

Lecture Homework Week 06 - Tuesday Template

Student Information

Name: Abhiram Kamiseti

NetID: akami006

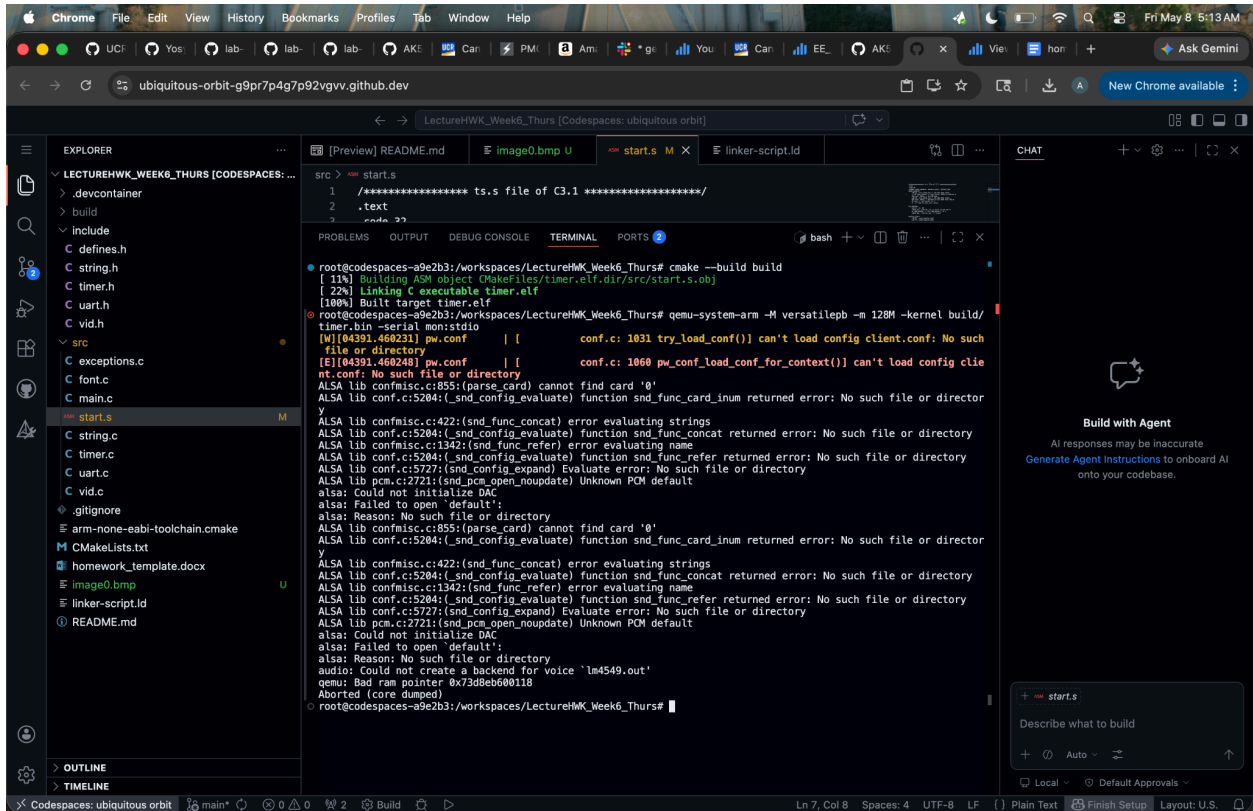
Student Responses

Answer the following questions, and be sure to highlight them when you turn them in on Gradescope

Problem 1

For part (1), below answer why the program does not work when you comment out the line BL copy_vector:

```
src > start.s
1 /***** ts.s file of C3.1 *****/
2 .text
3 .code 32
4 .global reset_handler, vectors_start, vectors_end
5 reset_handler:
6     LDR sp, =svc_stack_top // set SVC mode stack
7     // BL copy_vectors // copy vector table to address 0
8     MSR cpsr, #0x92 // to IRQ mode
9     LDR sp, =irq_stack_top // set IRQ mode stack
10    MSR cpsr, #0x13 // go back to SVC mode with IRQ on
11    BL main // call main() in C
12    B . // loop if main ever return
13
14 irq_handler:
15     sub lr, lr, #4
16     stmfd sp!, {r0-r12, lr} // stack r0-r12 and lr
17     bl IRQ_handler // call IRQ_handler() in C
18     ldmfd sp!, {r0-r12, pc}^ // return
19
20 vectors_start:
21     LDR PC, reset_handler_addr
22     LDR PC, undef_handler_addr
23     LDR PC, swi_handler_addr
24     LDR PC, prefetch_abort_handler_addr
25     LDR PC, data_abort_handler_addr
26     B .
27     LDR PC, irq_handler_addr
28     LDR PC, fiq_handler_addr
29     reset_handler_addr: .word reset_handler
30     undef_handler_addr: .word undef_handler
31     swi_handler_addr: .word swi_handler
32     prefetch_abort_handler_addr: .word prefetch_abort_handler
33     data_abort_handler_addr: .word data_abort_handler
34     irq_handler_addr: .word irq_handler
35     fiq_handler_addr: .word fiq_handler
36 vectors_end:
```



The ARM processor always checks 0x0 address when managing exceptions or interrupts. By commenting BL copy_vectors, the table is never copied from the 0x0 address so the processor discovers garbage values for data in the 0x0 address. This causes a bad memory address so it crashes

For part (2), below answer why moving the vector table to the beginning of the start.s file still works after recompiling:

